

Projet semestre 2

La bataille

Nom : Othmen Ghozia

Groupe : Licence 1 BI (2) Année : 2025 -
2026

Enseignant : M. Sofien MELKI

1. Introduction
2. Présentation du jeu
3. Approche et organisation
4. Structures de données
5. Fonctions principales
6. Difficultés rencontrées
7. Conclusion

- Ce projet consiste à implémenter le jeu de cartes La Bataille en langage C
- L'objectif principal est de pratiquer les structures de données dynamiques :
 - Les pointeurs
 - Les listes chaînées
 - La pile (Stack)
 - La file (Queue)
- Le projet a été réalisé individuellement sur une durée de 6 semaines

La bataille est un jeu de cartes à deux
joueurs :

- Un jeu de **52 cartes** est **mélangé** puis **distribué équitablement** : 26 cartes par joueur
- A chaque tour, chaque joueur retourne sa **première carte**
- La carte **la plus haute** remporte les deux cartes
 - En cas d'égalité : **BATAILLE !**
- Le joueur qui possède toutes les cartes **gagne**

Quand les deux cartes sont égales :

1. Chaque joueur pose 1 carte face cachée
2. Chaque joueur pose 1 carte face visible
3. La carte visible la plus haute remporte

Toutes les cartes posées

Si égalité encore → nouvelle bataille !

Si un joueur n'a plus de cartes → il perd

Le projet est divisé en 5 paires de fichiers :

carte.h → description d'une carte

liste.c/.h → liste chaînée (le paquet)

file.c/.h → file FIFO (main du joueur)

pile.c/.h → pile LIFO (zone de bataille)

jeu.c/.h → logique complète du jeu

main.c → point d'entrée du programme

```
typedef struct carte{  
    int valeur; //valeur de la carte  
    char couleur[10]; //couleur de la carte  
    struct carte *suivant; //pointe vers la carte suivante  
} carte;
```

- Chaque carte connaît la carte suivante
- C'est ce pointeur qui permet de former une liste chaînée

- tete pointe vers la première carte
- chaque carte pointe vers la suivante via
suivant
- La dernière carte pointe vers NULL
- Utilisée pour stocker le paquet de 52
cartes

fonction : melanger_jeu()

Algorithme de Fisher-Yates (tutoriel Youtube) :

1. Mettre tous les pointeurs dans un tableau
2. partir de la dernière case
3. Choisir une position aléatoire j
4. Echanger les cases i et j
5. Reculer d'une case et répéter

Avantage : chaque ordre est équiprobable
srand (time(NULL)) garantit un résultat différent à chaque exécution .

```

typedef struct {
    carte *tete;
    carte *queue;
    int taille;
} file;

void init_file(file *f);
void enfiler(file *f, carte *c);
carte *defiler (file *f);
int file_vide(file *f);
int taille_file(file *f);
void afficher_file(file *f);
void liberer_file(file *f);

```

init_file() → initialise une file vide

enfiler() → ajoute une carte en queue

defiler() → retire la carte en tete

file_vide() → vérifie si la file est vide

taille_file() → retourne le nombre de cartes

afficher_file → affiche la file

liberer_file() → libère toute la mémoire

```

typedef struct {
    carte *sommet; //top card
    int taille;
} pile;

void init_pile (pile *p); //initialisation d une pile vide
void empiler (pile *p, carte *c); //pose une carte au sommet
carte *depiler (pile *p); //retire et retourne top card
int pile_vide(pile *p); //pour verifier si la pile est vide ou non
int taille_pile (pile *p); //retourne le nb de cartes dans la pile
void vider_dans_file (pile *p, file *f); //
void liberer_pile (pile *p); //liberer la memoire
#endif // pile.h

```

init_pile() → initialise une pile vide

empiler() → pose une carte au la sommet

depiler() → retire la carte du sommet

pile_vide() → vérifie si la pile est vide

taille_pile() → retourne le nombre de cartes

vider_dans_file() → transfère toutes les
cartes vers la file du gagnant

liberer_pile() → libère toute la mémoire

fonction : distribuer()

- Parcourt le paquet mélangé carte par carte

- Alterne entre joueur 1 et joueur 2

carte 1 → joueur 1

carte 2 → Joueur 2

Carte 3 → Joueur 1 ...

Carte 52 → Joueur 2

Résultat : 26 cartes chacun

fonction : jouer_tour()

Joueur 1 joue sa première carte →
defiler()

Joueur 2 joue sa première carte →
defiler()

Comparaison des valeurs

Si $c1 > c2$ alors joueur 1 **gagne** les
deux cartes et enfiler() × 2 dans jo1

Si $c2 > c1$ alors joueur 2 **gagne** les
deux cartes et enfiler() × 2 dans jo2 Si
 $c1 == c2$ → bataille → gerer_bataille()

fonction : gerer_bataille()

1. Les deux cartes égales → empiler()
2. Chaque joueur pose 1 carte cachée →
empiler()
3. Chaque joueur révèle 1 carte visible
4. Comparaison des cartes visibles

GESTION DE LA MEMOIRE

(tutoriel youtube et avec l'aide l'IA)

Technique de libération **sécurisée** :

1. sauvegarder suivant avant de libérer
 2. Libérer la carte **courante**
 3. Avancer vers le suivant sauvegardé 4.
- Répéter jusqu'à **NULL**

Pourquoi utiliser ** dans certaines fonctions ?

(Encore avec l'aide de l'IA)

Règle simple :

- * (un étoile) → on **lit** la liste
- ** (deux étoiles) → on **modifie** où tete

pointe exemples :

`afficher_liste(carte *tete)` → lecture seule

`taille_liste(carte *tete)` → lecture seule

`insérer_queue(carte **tete)` → modifie tete

`libérer_liste(carte **tete)` → met tete à NULL

```

void choisir_vitesse(void) {
    int choix = 0;
    printf("\n");
    printf("*****\n");
    printf("*          CHOISISSEZ LA VITESSE          *\n");
    printf("*****\n");
    printf(" 1. Lent      (1 tour par seconde)      \n");
    printf(" 2. Rapide    (aussi vite que possible)  \n");
    printf(" 3. Manuel    (appuyez ENTREE pour avancer) \n");
    printf("\nVotre choix : ");
    scanf("%d", &choix);

    if (choix == 1) vitesse = 1;
    else if (choix == 2) vitesse = 2;
    else vitesse = 0;
}

void attendre(void) {
    if (vitesse == 0) {

        printf(" [Appuyez sur ENTREE pour continuer...]\n");
        getchar();
        getchar();
    } else if (vitesse == 1) {
        Sleep(1000);
    }
}

```

choix de la vitesse d'affichage

J'ai constaté que les manches avançaient trop rapidement , donc j'ai dû permettre à l'utilisateur de choisir une vitesse d'affichage selon ses préférences, mais c'était compliqué, alors j'ai dû recourir à des tutoriels sur YouTube

Problème rencontré :

Modifier tete depuis une fonction ne fonctionnait pas avec un seul *

Symptôme :

insérer_queue() ne modifiait pas la vraie liste, seulement une copie

Solution :

Utiliser ** pour passer l'adresse du pointeur
lui même insérer_queue(carte **tete, carte
*nouvelle)

Maintenant les modifications survivent en dehors de la fonction

Problème rencontré :

La bataille peut théoriquement ne jamais se terminer, les mêmes cartes peuvent circuler indéfiniment entre les joueurs

Symptôme :

Le jeu tourne toujours pendant des milliers de tours sans jamais se terminer

Solution :

Ajout d'une limite maximale de tours :

#define maxtours 10000

si cette limite est atteinte → le jeu déclare un match nul automatiquement

Ce projet m'a permis de maîtriser :

- Les **pointeurs** et l'arithmétique de **pointeurs**
- La création dynamique avec **malloc()** et **free()**
- La structure de **liste chaînée** et son parcours
- La différence entre **File (FIFO)** et **Pile (LIFO)**
 - L'utilisation de ****** pour modifier des pointeurs
 - La décomposition d'un problème complexe en fonctions indépendantes
- L'importance des commentaires et de la lisibilité
- L'algorithme de mélange **Fisher-Yates**

- Le jeu de La Bataille est entièrement fonctionnel et respecte toutes les contraintes du cahier des charges
- Les trois structures de données sont correctement implémentées
- Le bonus vitesse d'affichage est implémenté
- Ce projet m'a donné une solide base en structures de données dynamiques en C

```
creation du jeu de 52 cartes...
[paquet cree : 52 cartes.
melange du paquet...
distribution des cartes...
joueur 1 : 26 cartes | joueur 2 : 26 cartes
```

```
*****
*                JEU DE LA BATAILLE                *
*****
```

```
*****
*                CHOISISSEZ LA VITESSE                *
*****
```

1. Lent (1 tour par seconde)
2. Rapide (aussi vite que possible)
3. Manuel (appuyez ENTREE pour avancer)

```
Votre choix : |
```

```
*****
*                               *
*           FIN DE PARTIE      *
*                               *
*****
Nombre de tours joues : 1658
Cartes Joueur 1 : 52
Cartes Joueur 2 : 0

>>> VICTOIRE DU JOUEUR 1 !
```

```
*****
*                               *
*           FIN DE PARTIE      *
*                               *
*****
Nombre de tours joues : 642
Cartes Joueur 1 : 0
Cartes Joueur 2 : 52

>>> VICTOIRE DU JOUEUR 2 !
```

```
Process returned 0 (0x0)   execution time : 35.385 s
Press any key to continue.
```


MATCH NUL : limite de 10000 tours atteinte !

* FIN DE PARTIE *

Nombre de tours joues : 10000

Cartes Joueur 1 : 26

Cartes Joueur 2 : 26

Match nul.